

CDAC FUTURE SKILLS PROGRAMME

Demystifying Android Application Security

Day 03 – Assignment: Secure Coding Practices

Insecure Coding Practices, Secure Code Snippets and OWASP Mobile Top 10 Mapping

Name	: Vimal Mudalagi
Institution	: Digital University Kerala
Programme	: M.Sc. Computer Science
Specialization	: Cybersecurity

Task 1 – Identify Insecure Coding Practices

The application source code was reviewed using Android Studio / source-code inspection after downloading and extracting the provided project. Three major insecure coding practices were identified by searching the Kotlin source for security-relevant keywords, covering logging of sensitive data, unsafe WebView/SSL handling, weak cryptography, and Android manifest configuration.

Search commands used (PowerShell):

```
$src = Get-ChildItem .\app\src\main -Recurse -File -Include *.java,*.kt

"===== 1. LOGGING / SENSITIVE DATA ====="
$src | Select-String -Pattern 'Log.(d|e|i|v|w)|println|password|token|secret'
    | ForEach-Object { "{0}:{1}: {2}" -f $_.Filename, $_.LineNumber, $_.Line.Trim() }

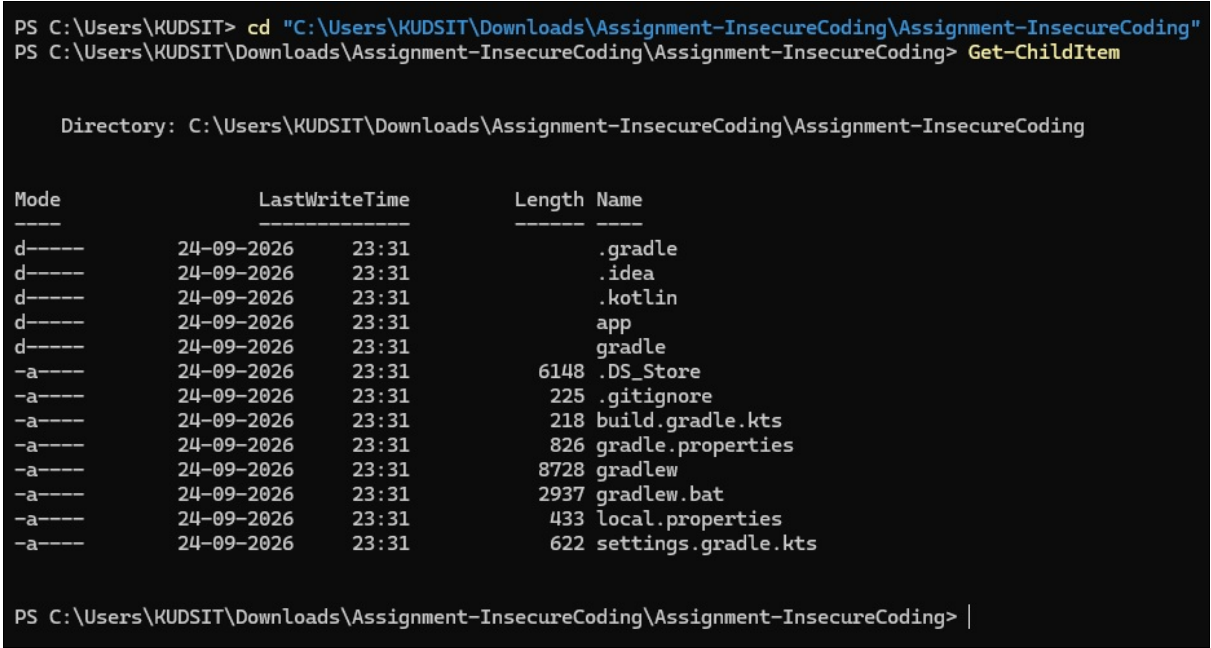
"===== 2. WEBVIEW / SSL ====="
$src | Select-String -Pattern
'WebView|setJavaScriptEnabled|onReceivedSslError|handler.proceed|loadUrl'
    | ForEach-Object { "{0}:{1}: {2}" -f $_.Filename, $_.LineNumber, $_.Line.Trim() }

"===== 3. CRYPTOGRAPHY ====="
$src | Select-String -Pattern 'Cipher|MessageDigest|MD5|SHA-1|DES|ECB|AES|SecretKey|KeyStore'
    | ForEach-Object { "{0}:{1}: {2}" -f $_.Filename, $_.LineNumber, $_.Line.Trim() }

"===== 4. ANDROID MANIFEST ====="
Get-Content ..\AndroidManifest.xml
    | Select-String -Pattern 'exported|debuggable|allowBackup|usesCleartextTraffic|permission'
    | ForEach-Object { $_.Line.Trim() }
```

Screenshot 1

Downloaded project extracted and opened in the working folder before running the searches.



Screenshot 2

Logging / sensitive-data keyword search results, showing password and token values being logged (e.g. LoginActivity.kt).

```

PS C:\Users\KUDSIT\Downloads\Assignment-InsecureCoding> $src = Get-ChildItem .\app\src\main -Recurse -File -Include *.java,*.kt
PS C:\Users\KUDSIT\Downloads\Assignment-InsecureCoding>
PS C:\Users\KUDSIT\Downloads\Assignment-InsecureCoding> $src | Select-String -Pattern 'Log\.(d|e|i|v|w)|println|password|token|secret' |
>> ForEach-Object {
>>     "{0}:{1}: {2}" -f $_.Filename, $_.LineNumber, $_.Line.Trim()
>> }
EncryptActivity.kt:26: import javax.crypto.spec.SecretKeySpec
EncryptActivity.kt:47: val keySpec = SecretKeySpec(HARDCODED_KEY.toByteArray(Charsets.UTF_8), "DES")
ExportedInsecureActivity.kt:24: private val adminToken = "internal-admin-token-000111222"
ExportedInsecureActivity.kt:43: Text("Admin Token: $adminToken")
LoginActivity.kt:49: private fun insecureLogin(context: Context, username: String, password: String): Boolean {
LoginActivity.kt:51: Log.d("LoginActivity", "Login attempt username=$username password=$password")
LoginActivity.kt:54: val query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'"
LoginActivity.kt:55: Log.d("LoginActivity", "Running query: $query")
LoginActivity.kt:61: .putString("last_password", password)
LoginActivity.kt:65: return username == HARDCODED_USER && password == HARDCODED_PASS
LoginActivity.kt:72: var password by remember { mutableStateOf("") }
LoginActivity.kt:90: value = password,
LoginActivity.kt:91: onValueChange = { password = it }, // no validation, no masking
LoginActivity.kt:92: label = { Text("Password") },
LoginActivity.kt:98: val success = insecureLogin(context, username, password)
WebViewActivity.kt:44: fun getDeviceData(): String = "device-id=EMULATOR-1234;secret=insecure-bridge-token"
PS C:\Users\KUDSIT\Downloads\Assignment-InsecureCoding>

```

Screenshot 3

WebView / SSL keyword search results.

```

PS C:\Users\KUDSIT\Downloads\Assignment-InsecureCoding> $src | Select-String -Pattern 'WebView|setJavaScriptEnabled|onReceivedSslError|handler\.\proceed|loadUrl' |
>> ForEach-Object {
>>     "{0}:{1}: {2}" -f $_.Filename, $_.LineNumber, $_.Line.Trim()
>> }
MainActivity.kt:79: onClick = { context.startActivity(Intent(context, WebViewActivity::class.java)) },
MainActivity.kt:82: Text("Load WebView")
WebViewActivity.kt:9: import android.webkit.WebView
WebViewActivity.kt:10: import android.webkit.WebViewClient
WebViewActivity.kt:16: class WebViewActivity : ComponentActivity() {
WebViewActivity.kt:19: "SetJavaScriptEnabled",
WebViewActivity.kt:21: "WebViewClientOnReceivedSslError"
WebViewActivity.kt:26: val webView = WebView(this)
WebViewActivity.kt:27: setContentView(webView)
WebViewActivity.kt:29: webView.settings.apply {
WebViewActivity.kt:42: webView.addJavascriptInterface(object : Any() {
WebViewActivity.kt:47: webView.webViewClient = object : WebViewClient() {
WebViewActivity.kt:48: override fun onReceivedSslError(
WebViewActivity.kt:49: view: WebView?,
WebViewActivity.kt:58: webView.loadUrl(url)

```

Screenshot 4

Cryptography keyword search results, showing DES/ECB usage in EncryptActivity.kt.

```
PS C:\Users\KUDSIT\Downloads\Assignment-InsecureCoding> $src | Select-String -Pattern 'Cipher|MessageDigest|MD5|SHA-1|DES|ECB|AES|SecretKey|KeyStore' |
>> ForEach-Object {
>>     "{0}:{1}: {2}" -f $_.Filename, $_.LineNumber, $_.Line.Trim()
>> }
Theme.kt:44: dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
EncryptActivity.kt:25: import javax.crypto.Cipher
EncryptActivity.kt:26: import javax.crypto.spec.SecretKeySpec
EncryptActivity.kt:47: val keySpec = SecretKeySpec(HARDCODED_KEY.toByteArray(Charsets.UTF_8), "DES")
EncryptActivity.kt:48: val cipher = Cipher.getInstance("DES/ECB/PKCS5Padding") // VULNERABLE: weak cipher + ECB
EncryptActivity.kt:49: cipher.init(Cipher.ENCRYPT_MODE, keySpec)
EncryptActivity.kt:50: val encrypted = cipher.doFinal(plainText.toByteArray(Charsets.UTF_8))
EncryptActivity.kt:66: Text("Uses DES/ECB with a hardcoded key.")
EncryptActivity.kt:83: Text("Ciphertext (Base64):")
```

Screenshot 5

AndroidManifest.xml keyword search results (exported, debuggable, allowBackup, permissions).

```
PS C:\Users\KUDSIT\Downloads\Assignment-InsecureCoding> Get-Content .\app\src\main\AndroidManifest.xml |
>> Select-String -Pattern 'exported|debuggable|allowBackup|usesCleartextTraffic|permission' |
>> ForEach-Object {
>>     $_.Line.Trim()
>> }
<!-- VULNERABLE: broad INTERNET permission, no scoping/network security config -->
<uses-permission android:name="android.permission.INTERNET" />
android:allowBackup="true"
android:usesCleartextTraffic="true"
android:exported="true"
<!-- VULNERABLE: exported="true" with no permission check lets any app on the
android:name=".ExportedInsecureActivity"
android:exported="true"
android:exported="true"
android:exported="true"
android:exported="true"
```

What these are doing

We're basically searching the source for:

- Sensitive data logging
- Unsafe WebView / SSL handling
- Weak/insecure cryptography
- Insecure Android components/configuration

Of the results found, the three practices below were used because they are extremely clear and easy to explain.

Summary of findings

#	Insecure practice	Evidence	OWASP Mobile Top 10 2024
1	SQL Injection	LoginActivity.kt:54	M4 – Insufficient Input/Output Validation
2	DES/ECB + hardcoded key	EncryptActivity.kt:47-48	M10 – Insufficient Cryptography
3	Exported activity without authorization	ExportedInsecureActivity + android:exported="true"	M3 – Insecure Authentication/Authorization

These mappings match the current OWASP Mobile Top 10 2024 categories.

Finding 1 – SQL Injection (LoginActivity.kt)

You found:

```
val query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'"
```

The application directly concatenates user-controlled `username` and `password` values into an SQL query. The problem is that user-controlled input is directly concatenated into an SQL query. An attacker could manipulate the input so the SQL query behaves differently from what the developer intended.

OWASP specifically identifies concatenating untrusted input into SQL queries instead of using parameterized APIs as an unsafe-data-handling weakness. **OWASP: M4 – Insufficient Input/Output Validation.**

Finding 2 – DES/ECB + hardcoded key (EncryptActivity.kt)

You found:

```
val keySpec = SecretKeySpec(
    HARDCODED_KEY.toByteArray(Charsets.UTF_8),
    "DES"
)
val cipher = Cipher.getInstance("DES/ECB/PKCS5Padding")
```

This is actually two problems in one:

- DES is an outdated/weak encryption algorithm.
- ECB is an insecure encryption mode for general sensitive-data encryption.
- The encryption key is also hardcoded in the application.

OWASP's current mobile guidance classifies broken/risky cryptographic algorithms and insecure encryption modes under improper encryption/insufficient cryptography. **OWASP: M10 – Insufficient Cryptography.**

Finding 3 – Exported Activity (ExportedInsecureActivity.kt)

We have:

```
android:exported="true"
```

and, in `ExportedInsecureActivity.kt`, the source also contains:

```
private val adminToken = "internal-admin-token-000111222"
```

This is particularly notable because the component is exposed to other applications, while the code appears to contain privileged/admin functionality. OWASP specifically describes exported Android components without adequate authentication/authorization as a weakness that can allow other apps to invoke sensitive functionality. **OWASP: M3 – Insecure Authentication/Authorization.**

Source code listings reviewed

The following commands were used to view the full contents of the three files containing the identified insecure practices:

```
Get-ChildItem .\app\src\main -Recurse -Filter LoginActivity.kt | Get-Content
Get-ChildItem .\app\src\main -Recurse -Filter EncryptActivity.kt | Get-Content
Get-ChildItem .\app\src\main -Recurse -Filter ExportedInsecureActivity.kt | Get-Content
```

LoginActivity.kt

Full source listing (part 1 of 2).

```
PS C:\Users\KUDSIT\Downloads\Assignment-InsecureCoding> Get-ChildItem .\app\src\main -Recurse -Filter LoginActivity.kt | Get-Content
package com.example.insecureapp

import android.content.Context
import android.os.Bundle
import android.util.Log
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import com.example.insecureapp.ui.theme.InsecureAppTheme

/**
 * FIND vulnerable or insecure coding practice being followed
 */
class LoginActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            InsecureAppTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    LoginScreen(modifier = Modifier.padding(innerPadding))
                }
            }
        }
    }
}

private const val HARDCODED_USER = "admin"
private const val HARDCODED_PASS = "admin123"
```

```
.putString("last_password", password)
.apply()

return username == HARDCODED_USER && password == HARDCODED_PASS
}

@Composable
fun LoginScreen(modifier: Modifier = Modifier) {
    val context = LocalContext.current
    var username by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }

    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text("Insecure Login", style = MaterialTheme.typography.headlineSmall)

        OutlinedTextField(
            value = username,
            onChange = { username = it }, // no validation/sanitization
            label = { Text("Username") },
            modifier = Modifier.fillMaxWidth()
        )

        OutlinedTextField(
            value = password,
            onChange = { password = it }, // no validation, no masking
            label = { Text("Password") },
            modifier = Modifier.fillMaxWidth()
        )

        Button(
            onClick = {
                val success = insecureLogin(context, username, password)
                Toast.makeText(
                    context,
                    if (success) "Login successful" else "Login failed",
                    Toast.LENGTH_SHORT
                ).show()
            },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Login")
        }
    }
}
```

EncryptActivity.kt

Full source listing (part 1 of 2).

```

PS C:\Users\KUDSIT\Downloads\Assignment-InsecureCoding> Get-ChildItem .\app\src\main -Recurse -Filter EncryptActivity.kt | Get-Content
package com.example.insecureapp

import android.os.Bundle
import android.util.Base64
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.insecureapp.ui.theme.InsecureAppTheme
import javax.crypto.Cipher
import javax.crypto.spec.SecretKeySpec

/**
 * FIND Vulnerable or insecure way of encryption
 */
class EncryptActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            InsecureAppTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    EncryptScreen(modifier = Modifier.padding(innerPadding))
                }
            }
        }
    }
}

private const val HARDCODED_KEY = "8bytekey"

private fun insecureEncrypt(plainText: String): String {

```



```
private const val HARDCODED_KEY = "8bytekey"

private fun insecureEncrypt(plainText: String): String {
    val keySpec = SecretKeySpec(HARDCODED_KEY.toByteArray(Charsets.UTF_8), "DES")
    val cipher = Cipher.getInstance("DES/ECB/PKCS5Padding") // VULNERABLE: weak cipher + ECB
    cipher.init(Cipher.ENCRYPT_MODE, keySpec)
    val encrypted = cipher.doFinal(plainText.toByteArray(Charsets.UTF_8))
    return Base64.encodeToString(encrypted, Base64.DEFAULT)
}

@Composable
fun EncryptScreen(modifier: Modifier = Modifier) {
    var input by remember { mutableStateOf("") }
    var output by remember { mutableStateOf("") }

    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text("Insecure Encryption", style = MaterialTheme.typography.headlineSmall)
        Text("Uses DES/ECB with a hardcoded key.")

        OutlinedTextField(
            value = input,
            onValueChange = { input = it },
            label = { Text("Text to encrypt") },
            modifier = Modifier.fillMaxWidth()
        )

        Button(
            onClick = { output = insecureEncrypt(input) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Encrypt")
        }

        if (output.isNotEmpty()) {
            Text("Ciphertext (Base64):")
            Text(output)
        }
    }
}
```

ExportedInsecureActivity.kt

Full source listing.

```

PS C:\Users\KUDSIT\Downloads\Assignment-InsecureCoding> Get-ChildItem .\app\src\main -Recurse -Filter ExportedInsecureActivity.kt | Get-Content
package com.example.insecureapp

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.insecureapp.ui.theme.InsecureAppTheme

/**
 * Find vulnerable or insecure coding practice being used
 */
class ExportedInsecureActivity : ComponentActivity() {

    private val apiKey = "sk-live-INSECURE-51df9c3a7b2e4410"
    private val adminToken = "internal-admin-token-000111222"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            InsecureAppTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Column(
                        modifier = Modifier
                            .padding(innerPadding)
                            .padding(24.dp),
                        verticalArrangement = Arrangement.spacedBy(12.dp)
                    ) {
                        Text(
                            text = "Insecure Activity",
                            style = MaterialTheme.typography.headlineSmall
                        )
                        Text("This screen is exported and unprotected. Any app can reach it.")
                        Text("API Key: $apiKey")
                        Text("Admin Token: $adminToken")
                    }
                }
            }
        }
    }
}

```

Task 1 complete.

Task 2 – Secure Code Snippets

1. Secure SQL Query – LoginActivity.kt

Insecure code

```
val query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'"
```

Secure version

Use a **parameterized query** instead of directly concatenating user input:

```
val db = dbHelper.readableDatabase

val cursor = db.query(
    "users",
    arrayOf("username"),
    "username = ? AND password = ?",
    arrayOf(username, password),
    null,
    null,
    null
)

val success = cursor.moveToFirst()
cursor.close()
```

Why this is secure: The `?` placeholders separate the SQL command from the user-supplied values, preventing the input from being interpreted as SQL code.

For a real application, passwords should also be **properly hashed**, not stored/compared as plaintext. For this assignment, the important fix here is **parameterized SQL**.

2. Secure Encryption – EncryptActivity.kt

Insecure code

```
private const val HARDCODED_KEY = "8bytekey"

val keySpec = SecretKeySpec(
    HARDCODED_KEY.toByteArray(Charsets.UTF_8),
    "DES"
)

val cipher = Cipher.getInstance("DES/ECB/PKCS5Padding")
```

Secure version

Use **AES-GCM** with a securely generated key and a random IV:

```
val keyGenerator = KeyGenerator.getInstance("AES")
keyGenerator.init(256)
val secretKey = keyGenerator.generateKey()

val cipher = Cipher.getInstance("AES/GCM/NoPadding")

val iv = ByteArray(12)
SecureRandom().nextBytes(iv)

val spec = GCMParameterSpec(128, iv)
cipher.init(Cipher.ENCRYPT_MODE, secretKey, spec)
```

```
val encrypted = cipher.doFinal(
    plainText.toByteArray(Charsets.UTF_8)
)
```

Required imports:

```
import java.security.SecureRandom
import javax.crypto.Cipher
import javax.crypto.KeyGenerator
import javax.crypto.spec.GCMParameterSpec
```

Why this is secure: AES-GCM provides authenticated encryption, avoids the weaknesses of DES/ECB, and uses a random IV instead of a fixed hardcoded encryption key.

For a production Android application, the generated key should preferably be stored using the **Android Keystore** rather than simply kept in application memory.

3. Secure Handling of API Keys / Admin Tokens – ExportedInsecureActivity.kt

Insecure code

```
private val apiKey = "sk-live-INSECURE-51df9c3a7b2e4410"
private val adminToken = "internal-admin-token-000111222"
```

Secure approach

Do not hardcode real API keys or administrator tokens inside the APK. For example:

```
private fun getApiData(token: String) {
    // Use a token obtained securely at runtime.
    // Do not store permanent secrets directly in the APK.
    makeAuthenticatedRequest(token)
}
```

And protect the Activity from unauthorized applications in the manifest:

```
<activity
    android:name=".ExportedInsecureActivity"
    android:exported="false" />
```

If the Activity genuinely needs to be accessed by another application, use an appropriate **permission and authentication/authorization check** instead of leaving it openly exported.

Why this is secure: Secrets are not embedded directly in the application source/APK, and an Activity that does not need external access is prevented from being launched by other applications.

Task 2 Summary

Insecure practice	Secure coding practice
SQL string concatenation	Parameterized SQL query
DES/ECB + hardcoded key	AES-GCM + secure key generation + random IV
Hardcoded credentials + unprotected Activity	Secure runtime secret handling + <code>exported="false"</code>

Task 2 complete.

Task 3 – Associated Vulnerabilities

1. SQL Injection

Insecure coding practice: The application directly concatenates the `username` and `password` values into an SQL query.

```
val query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'"
```

The problem is that user-controlled input is directly concatenated into an SQL query. An attacker could manipulate the input so the SQL query behaves differently from what the developer intended. OWASP specifically identifies concatenating untrusted input into SQL queries instead of using parameterized APIs as an unsafe-data-handling weakness.

OWASP: M4 – Insufficient Input/Output Validation.

2. DES/ECB + hardcoded key

Insecure coding practice: The application uses a hardcoded DES key together with the insecure ECB encryption mode.

```
private const val HARDCODED_KEY = "8bytekey"

val keySpec = SecretKeySpec(
    HARDCODED_KEY.toByteArray(Charsets.UTF_8),
    "DES"
)

val cipher = Cipher.getInstance("DES/ECB/PKCS5Padding")
```

This is insecure because DES is an outdated and weak encryption algorithm, while ECB mode does not provide adequate protection for structured data. The encryption key is also hardcoded inside the application, making it easier to extract.

OWASP: M10 – Insufficient Cryptography.

3. Exported and Unprotected Activity

Insecure coding practice: The application contains an activity intended to be externally accessible without proper protection. The source itself states:

```
Text("This screen is exported and unprotected. Any app can reach it.")
```

The activity also contains sensitive credentials:

```
private val apiKey = "[REDACTED]"
private val adminToken = "[REDACTED]"
```

The corresponding Android manifest declares the component with:

```
android:exported="true"
```

An exported component without appropriate authentication or authorization controls can potentially be accessed by other applications on the device. Additionally, storing sensitive credentials directly in application source code exposes them to anyone who can inspect or

reverse-engineer the application.

OWASP: M3 – Insecure Authentication/Authorization.

Task 3 – Summary Table

#	Insecure practice	Vulnerability introduced	OWASP Mobile Top 10 2024
1	SQL string concatenation (LoginActivity.kt)	SQL Injection – attacker-controlled input can alter query logic and potentially bypass authentication or access unauthorized data	M4 – Insufficient Input/Output Validation
2	DES/ECB + hardcoded key (EncryptActivity.kt)	Weak/broken encryption – encrypted data can potentially be decrypted or pattern-analyzed due to ECB mode and a recoverable hardcoded key	M10 – Insufficient Cryptography
3	Exported activity with hardcoded credentials (ExportedInsecureActivity.kt)	Insecure authorization – any application on the device can launch the activity and potentially reach privileged/admin functionality and embedded secrets	M3 – Insecure Authentication/Authorization

Task 3 complete.